Präventatives Defogging zum Steigern der Energieeffizienz

T1000

für die

T1000

des Studiengangs Informatik

an der Dualen Hochschule Baden-Württemberg Stuttgart

von

Viktoria Backhaus

Dezember 2025

Bearbeitungszeitraum	29.10.2025 - 18.12.2025
Matrikelnummer, Kurs	1695387, T41000
Ausbildungsfirma	Mercedes-Benz AG, Stuttgart
Betreuer	Paul Karcher B. Eng
Gutachter	

Sperrvermerk

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anderslautende Genehmigung der Ausbildungsstätte vorliegt.

Stuttgart, Dezember 2025

Viktoria Backhaus

Erklärung

Ich versichere hiermit, dass ich meine T1000 mit dem Thema: *Präventatives Defogging zum Steigern der Energieeffizienz* selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Stuttgart, Dezember 2025

Viktoria Backhaus

Inhaltsverzeichnis

Abkürzungsverzeichnis **IV**

Abbildungsverzeichnis **V**

1 Einleitung **1**

2 Theorieteil **2**
2.1 Mikrocontroller 2
2.1.1 Was ist ein Mikrocontroller? 2
2.1.2 Besonderheiten des ESP32 2
2.2 Der DHT11 Sensor 3
2.3 Kommunikation und OSI-Modell 3
2.3.1 Physikalische Schicht / Bitübertragungsschicht 3
2.3.2 Datenverbindungsschicht / Sicherungsschicht 4
2.3.3 Netzwerkschicht / Vermittlungsschicht 4
2.3.4 Transportschicht 4
2.3.5 Applikationsschicht 5
2.4 Stand der Technik 5
2.4.1 Aktuelle Methoden zum Defogging 5
2.4.2 Temperatur und Feuchtigkeitssensoren 6

3 Umsetzung **7**
3.1 Planung und Aufbau 7
3.2 Durchführung 8
3.2.1 Messreihe zur Genauigkeit des DHT11 Sensors 8
3.3 Theoretische Implementierung zum Defogging 9
3.4 Kritische Betrachtung und Lösungsansätze 10
3.4.1 Lösungsansatz 1: Aktivierung nur beim Ladevorgang 10
3.4.2 Lösungsansatz 2: Intelligentes Defogging 11
3.4.3 Lösungsansatz 3: Ferngesteuertes Defogging 11

4 Fazit und Ausblick **12**

Literatur **13**

Anhang **14**
A. Code 15

Abkürzungsverzeichnis

HVAC Heating, Ventilation and Air Conditioning

MCU Mikrocontroller-Einheit

EEPROM electrically erasable programmable read-only memory

IoT Internet of Things

OSI Open Systems Interconnection

TCP Transmission Control Protocol

UDP User Datagram Protocol

Abbildungsverzeichnis

3.1 Schaltplan DHT11 (eigene Abbildung) 7
3.2 Messungen zur Genauigkeit des DHT11 Sensors 8
3.3 Ausschnitt aus der Taupunkttabelle ([7], S.1) 9

4.1 Anhang: Schaltplan DHT11 (eigene Abbildung) 17
4.2 Anhang: Luftfeuchtigkeitsmessreihe 18
4.3 Anhang:Temperaturmessreihe 19
4.4 Anhang: Auschnitt aus der Taupunkttabelle ([7], S.1) 19

1 Einleitung

Beschlagene Windschutzscheiben stellen ein Sicherheitsrisiko dar und erhöhen gleichzeitig den Energiebedarf von Fahrzeugen. Besonders in Elektrofahrzeugen kann der Einsatz des Heiz-, Lüftungs- und Klimasystems Heating, Ventilation and Air Conditioning (HVAC) System die Reichweite um bis zu 13% reduzieren (vgl. [10]). Da die Enteisung oder Entfeuchtung der Scheibe meist reaktiv erfolgt, entsteht kurzfristig ein hoher Energieverbrauch.

Ein präventiver Ansatz könnte diesen Bedarf reduzieren: Wenn Temperatur- und Feuchtigkeitssensoren frühzeitig erkennen, dass die Bedingungen für eine Kondensation erreicht werden, lassen sich geeignete Maßnahmen bereits vor dem Beschlag einleiten.

Ziel dieser Arbeit ist es daher, das Potenzial eines auf ESP32 und DHT11 basierenden Sensorsystems zur frühzeitigen Erkennung von Beschlagbedingungen zu untersuchen und dessen Eignung im Hinblick auf Energieeffizienz zu bewerten.

2 Theorieteil

2.1 Mikrocontroller

2.1.1 Was ist ein Mikrocontroller?

Eine Mikrocontroller-Einheit (MCU) ist ein kleiner Computer, bestehend aus nur einem Chip. Sie wurden speziell dafür entwickelt, um Aufgaben in Systemen zu übernehmen, ohne dafür ein komplexes Betriebssystem zu benötigen.

Er enthält einen Prozessor, Arbeitsspeicher und einen programmierbaren Speicher (electrically erasable programmable read-only memory (EEPROM)) sowie Ein- und Ausgabeschnittstellen. Im Gegensatz zu Mikroprozessoren benötigt ein Mikrocontroller keine weiteren externen Komponenten. Außerdem sind sie optimiert für die Steuerung von Sensoren oder Motoren(vgl. [9]).

2.1.2 Besonderheiten des ESP32

Der ESP32 ist ein leistungsfähiger Mikrocontroller und der Nachfolger des ESP8266 von Espressif. Er verfügt über einen Dual-Core 32-Bit Prozessor mit zwei Kernen, was ihn besonders schnell macht.

WLAN gehört zu seinen Kernfunktionen. Der ESP32 kann sich mit WLAN-Netzwerken verbinden und zusätzlich ein eigenes WLAN-Netzwerk (Access Point) aufbauen, beispielsweise als kleiner Webserver. Zudem unterstützt er Bluetooth, was ihn für Smart-Home- und Internet of Things (IoT)-Anwendungen ideal macht.

Der Mikrocontroller überzeugt auch durch einen sehr geringen Stromverbrauch, der im Deep-Sleep-Modus je nach Board unter 0,1 mA liegen kann, was ihn besonders für energieeffiziente IoT-Projekte prädestiniert.

Der ESP32 ist robust und kann in einem Temperaturbereich von -40 °C bis +125 °C zuverlässig betrieben werden.

Entwicklungsboards sind so aufgebaut, dass sie neben solchen Modulen auch die notwendige Versorgung und Anschlüsse bereitstellen, um die vielfältigen Funktionen des ESP32 einfach nutzen zu können (vgl. [4]).

2.2 Der DHT11 Sensor

Der DHT11 Sensor ist in der Lage, Temperatur und Luftfeuchtigkeit zu messen.

Zum Messen der Temperatur hat der DHT11 ein Negative Temperature Coefficient Thermistor. Dabei handelt es sich um einen Widerstand, welcher geringer wird je höher die Temperatur ist, da die Leitfähigkeit des Materials zunimmt.

Für die Luftfeuchtigkeit ist ein Kondensator in den DHT11 eingebaut. Dieser besteht aus zwei Elektroden, zwischen diesen befindet sich ein flüssigkeitsempfindliches Substrat. Je höher die Luftfeuchtigkeit ist, desto mehr Kapazität besitzt dieses Substrat.

Aus diesen Daten ermittelt der Sensor die tatsächlichen Werte durch eine Kalibrierung, welche durch den Hersteller vorgenommen wurde.

Er ist in der Lage Temperaturen von 0°C bis 50°C bei einer Messgenauigkeit von ± 2,0°C zu erfassen die Luftfeuchtigkeit kann im Bereich von 20% bis 90% gemessen werden, wobei die Messgenauigkeit ± 5% beträgt (vgl. [5]).

2.3 Kommunikation und OSI-Modell

Das Open Systems Interconnection (OSI)-Modell ist ein konzeptionelles Schichtenmodell zur Standardisierung der Netzwerkkommunikation. Es teilt die Kommunikation in 7 klar abgegrenzte abstrakte Ebenen ein, die jeweils spezifische Funktionen erfüllen und miteinander über definierte Schnittstellen interagieren (vgl. [8]). In vielen Fällen wird das OSI-Modell auch vereinfacht mit 5 Schichten dargestellt wird, wobei die Funktionen der Sitzungsschicht, Darstellungsschicht und Anwendungsschicht in einer einzigen Anwendungsschicht zusammengefasst werden (vgl. [6]).

Die sieben Schichten sind:

2.3.1 Physikalische Schicht / Bitübertragungsschicht

„Die physikalische Schicht regelt, wie die Bits zwischen miteinander direkt verbundenen Stationen physikalisch übertragen werden.“ ([6], S.130). Die physikalischen Medien sind Stecker, Router, Kabel sowie aber auch WLAN (vgl. [8], S.1), welches in diesem Projekt die Bits vom ESP32 auf das Mobiltelefon überträgt.

2.3.2 Datenverbindungsschicht / Sicherungsschicht

Diese Schicht liegt über der Physikalischen Schicht. Ihre Hauptaufgabe besteht darin, den Datenaustausch zweier miteinander verbundenen Geräte fehlerfrei ablaufen zu lassen.

„ Die auf dieser Schicht übertragenen Informationseinheiten werden auch Datenrahmen (kurz: Rahmen) oder Frames genannt“([6], S.130).Aufgrund der physikalischen Eigenschaften des verwendeten Übertragungsmediums sowie weiterer technischer Faktoren sind diese Rahmen häufig in ihrer Länge eingeschränkt oder müssen eine bestimmte Mindestgröße besitzen (vgl. [6], S.130).

Die Sicherungsschicht steuert, wer wann senden darf (Zugriffssteuerung) und erkennt Übertragungsfehler, um Korrekturmaßnahmen einzuleiten (vgl. [8]).

2.3.3 Netzwerkschicht / Vermittlungsschicht

Aufgabe dieser Schicht ist das Routing, also das Weiterleiten der Datenpakete innerhalb eines Netzwerks. Auf dieser Schicht werden die Daten "Pakete"genannt. Falls nötig werden die Pakete hier fragmentiert, also in kleinere Pakete zerlegt, falls es nötig sein sollte (vgl. [6], S.130).

2.3.4 Transportschicht

Diese Schicht stellt die zuverlässige Ende-zu-Ende-Kommunikation sicher und segmentiert größere Datenströme. Sie verwendet typischerweise Protokolle wie TCP und UDP:

Transmission Control Protocol (TCP): Verbindungsorientiert, garantiert fehlerfreie und geordnete Datenübertragung. Wichtig, wenn Datenverlust oder Reihenfolge vermieden werden muss.

User Datagram Protocol (UDP): Verbindungslos, schneller, aber ohne Garantie für Zustellung oder Reihenfolge. Geeignet, wenn Geschwindigkeit wichtiger ist als absolute Zuverlässigkeit.

2.3.5 Applikationsschicht

Sitzungsschicht: Verwalten von Kommunikationssitzungen zwischen Anwendungen, einschließlich Aufbau, Verwaltung und Beendigung von Verbindungen (vgl. [8]).

Darstellungsschicht: Zuständig für die Übersetzung und Formatierung von Daten, Verschlüsselung sowie Datenkompression, damit die Daten von der Anwendungsschicht verstanden werden können. Unterstützt z. B. SSL/TLS zur sicheren Datenübertragung (vgl. [8]).

Anwendungsschicht: Die oberste Schicht, die direkt mit Softwareanwendungen interagiert. Sie stellt Protokolle bereit für Dienste wie E-Mail (SMTP), Dateiübertragung (FTP) oder Webzugriff (HTTP). Im Projekt könnten hier Protokolle wie MQTT zur Sensordatenübertragung eingesetzt werden (vgl. [9]).

2.4 Stand der Technik

2.4.1 Aktuelle Methoden zum Defogging

Der Beschlag von Fahrzeugscheiben ist eine sicherheitskritische Folge der Kondensation von Wasserdampf auf kühleren Glasoberflächen; als Phasenwechselprozess erfordert seine Beseitigung zusätzliche Energie und beeinflusst Fahrkomfort und Sicht maßgeblich. Besonders in Elektrofahrzeugen erhöht der Betrieb des HVAC-Systems den Energiebedarf deutlich und kann die Reichweite um bis zu etwa 13% reduzieren, weshalb effiziente Defoggingstrategien zentral sind (vgl. [3], S.1-2).

Der Stand der Technik umfasst drei primäre Verfahren: (1) Erwärmte und gerichtete Luftzufuhr über das HVAC-System zur Trocknung der Grenzschicht an der Windschutzscheibe, (2) elektrische Widerstandsheizungen im Verbundglas zur schnellen lokalen Erwärmung und (3) den Einsatz der Klimaanlage zur Luftentfeuchtung durch Abkühlung unter Taupunkt mit anschließender Erwärmung der Zuluft; Hybridlösungen kombinieren Luftführung und elektrische Heizer, um Wirkung und Energieeinsatz auszubalancieren (vgl. [3], S.1-2).

Nach EU-Richtlinie 78/317/EWG muss im Sichtfeld vor dem Fahrenden ein Bereich A innerhalb von 10 Minuten zu 90% entnebelt sein; ein größerer Bereich B ist innerhalb von 10 Minuten zu 80% zu entnebeln; ähnliche Vorgaben existieren in Australien und den USA (FMVSS 103) (vgl. [3], S.2).

2.4.2 Temperatur und Feuchtigkeitssensoren

Neben dem DHT11 existieren zahlreiche weitere Sensoren zur Messung von Temperatur und Luftfeuchtigkeit. Der DHT22 stellt das Nachfolgermodell des DHT11 dar und bietet einen erweiterten und genaueren Messbereich. „Der Temperaturmessbereich reicht von -40°C bis 80°C und der Messbereich der Luftfeuchtigkeit reicht von 0% bis 100% mit einer Genauigkeit bzw. Abweichung von ±0,5°C und ±2-5%“ ([2]).

Alternativ können standardisierte Widerstandsthermometer wie Pt100 oder Ni100 eingesetzt werden. Diese Sensoren besitzen deutlich größere Messbereiche (Ni100: -60°C bis 180°C, Pt100: -200°C bis zu 850°C) (vgl. [1], S.233-236). Für das präventive Defogging in Fahrzeugen ist dieser große Bereich jedoch nicht notwendig, da lediglich Innen- und Außentemperaturen relevant sind. Zudem erfassen diese Sensoren ausschließlich die Temperatur, weshalb zur Bestimmung des Taupunkts ein zusätzlicher Feuchtigkeitssensor erforderlich wäre.

Der DHT11 ist daher für die Anwendung am besten geeignet. Sein Messbereich entspricht den typischen Werten, die in Taupunkttabellen (Abbildung 3.3) verwendet werden. Diese Tabellen zeigen die Temperatur an, bei der Kondensation entsteht, abhängig von Lufttemperatur und relativer Luftfeuchtigkeit. Relevante Temperaturwerte liegen meist zwischen 2°C und 50°C, die Feuchtigkeit zwischen 45% und 95%.

3 Umsetzung

3.1 Planung und Aufbau

Für den praktischen Aufbau des Systems werden folgende Komponenten benötigt:

- DHT11 - Temperatur- und Luftfeuchtigkeitssensor
- ESP32 - Mikrocontroller mit WLAN-Funktion
- Breadboard - zur schnellen, lötfreien Prototypenentwicklung
- Micro-USB-Kabel - zur Stromversorgung und Programmierung
- 3 Jumper-Kabel - zur elektrischen Verbindung zwischen Sensor und Mikrocontroller

Der Aufbau erfolgt wie in Abbildung 3.1 dargestellt. Der DHT11 wird mit drei Leitungen an den ESP32 angeschlossen: ein Datenpin, ein 3.3-V-Anschluss sowie eine Masseverbindung. Das Micro-USB-Kabel verbindet den ESP32 mit einem Computer und dient sowohl als Stromquelle als auch zur Übertragung des Programmcodes. Durch die geringe Leistungsaufnahme des Systems reicht die USB-Stromversorgung vollständig aus.

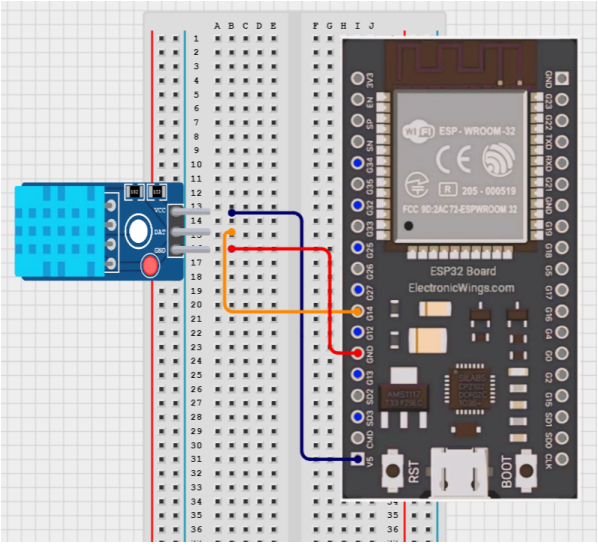


Abbildung 3.1: Schaltplan DHT11 (eigene Abbildung)

Zur Programmierung wird Visual Studio Code mit der Erweiterung PlatformIO genutzt, die Mikrocontrollerprojekte, Bibliothekenverwaltung, Kompilieren und Hochladen auf den

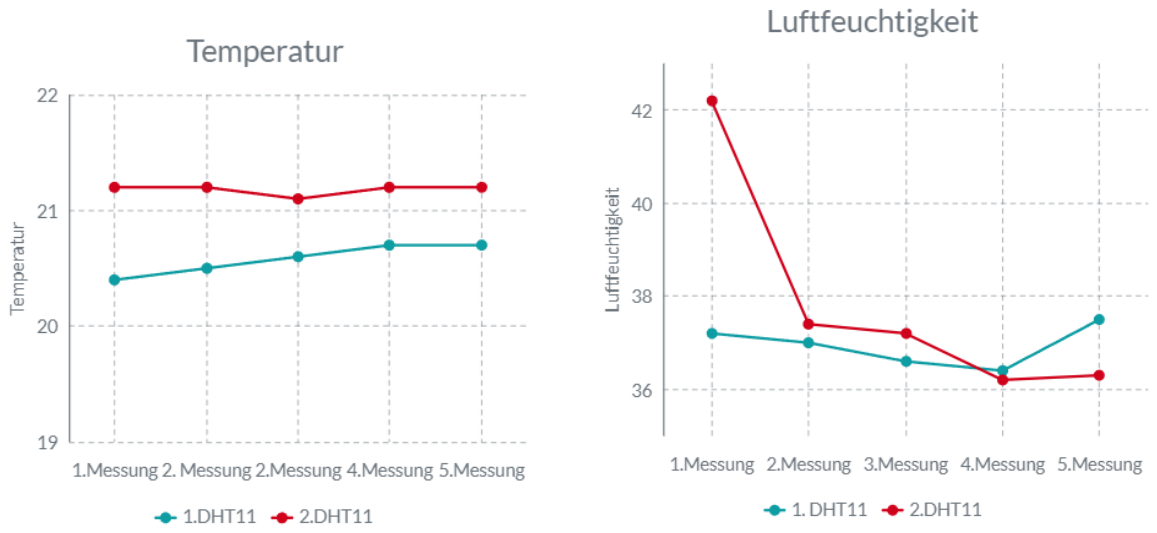
ESP32 vereinfacht. Für eine einfache Weboberfläche zur Anzeige der Messdaten kommt zusätzlich Live Server zum Einsatz, um die Website lokal zu hosten und in Echtzeit zu aktualisieren.

Der Plan sieht vor, den DHT11 in regelmäßigen Abständen auszulesen, die Temperatur- und Feuchtigkeitswerte digital zu verarbeiten und über den seriellen Monitor zu prüfen. Optional können die Daten über die Weboberfläche dargestellt werden, um die Genauigkeit des Sensors für Anwendungen wie Defogging-Systeme zu beurteilen.

3.2 Durchführung

3.2.1 Messreihe zur Genauigkeit des DHT11 Sensors

Für die Durchführung der Messungen wurde der Versuchsaufbau in einem geschlossenen Raum platziert, um konstante Umgebungsbedingungen zu gewährleisten. Dann wurden je fünf Messungen mit zwei verschiedenen DHT11 Sensoren gestartet, um sicher zu gehen, dass der verwendete Sensor nicht durch einen FEertigungsfehler oder Beschädigung ungenauer ist als andere DHT11 Sensoren. Der ESP32 liest den DHT11 in einem festen Intervall von zwei Sekunden aus. Die Messwerte werden unmittelbar über die serielle Schnittstelle ausgegeben und auf der vom ESP32 gehosteten Website ausgegeben.



(a) Messreihe Temperatur (eigene Abbildung) (b) Messreihe Luftfeuchtigkeit (eigene Abbildung)

Abbildung 3.2: Messungen zur Genauigkeit des DHT11 Sensors

Die Standardabweichung der Temperaturmessungen über beide Sensoren liegt bei 0,32°C, während die Luftfeuchtigkeit eine Standardabweichung von 1,75% aufweist. Diese Werte deuten darauf hin, dass die Temperatursensoren eine hohe Präzision aufweisen, während die Feuchtigkeitssensoren geringfügig stärkeren Schwankungen unterliegen.

Ein Vergleich der beiden Sensoren zeigt, dass Sensor 2 systematisch leicht höhere Werte sowohl für Temperatur (+0,56°C) als auch für Luftfeuchtigkeit (+0,92%) liefert. Was jedoch ebenfalls darauf zurückzuführen sein konnte, dass sowohl die Luftfeuchtigkeit als auch die Temperatur im Raum gestiegen ist, da sich mehrere Menschen dort befanden.

Insgesamt bestätigen die Daten jedoch eine zufriedenstellende Übereinstimmung der Sensoren. Die geringe Streuung der Messwerte spricht für eine zuverlässige Messgenauigkeit, die für den Einsatz im präventiven Defogging geeignet ist.

3.3 Theoretische Implementierung zum Defogging

Im Folgenden wird beschrieben, wie das entwickelte Sensorsystem theoretisch zur Prävention von Scheibenbeschlag eingesetzt werden kann. Grundlage hierfür ist das physikalische Prinzip der Kondensation: Beschlag entsteht, wenn die Temperatur der Scheibenoberfläche unter den Taupunkt der umgebenden Luft fällt (vgl. [3], S.2). Zur Bestimmung dieses Zusammenhangs werden Taupunkttabellen herangezogen.

Luft-Temp.	Taupunkttemperaturen in °C bei einer relativen Luftfeuchtigkeit von										
C°	45%	50%	55%	60%	65%	70%	75%	80%	85%	90%	95%
2°C	-7.77	-6.56	-5.43	-4.40	-3.16	-2.48	-1.77	-0.98	-0.26	0.47	1.20
4°C	-6.11	-4.88	-3.69	-2.61	-1.79	-0.88	-0.09	0.78	1.62	2.44	3.20
6°C	-4.49	-3.07	-2.10	-1.05	-0.08	0.85	1.86	2.72	3.62	4.48	5.38
8°C	-2.69	-1.61	-0.44	0.67	1.80	2.83	3.82	4.77	5.66	6.48	7.32
10°C	-1.26	0.02	1.31	2.53	3.74	4.79	5.82	6.79	7.65	8.45	9.31

Abbildung 3.3: Ausschnitt aus der Taupunkttabelle ([7], S.1)

Die Taupunkttabelle gibt an, bei welchen Kombinationen aus Lufttemperatur und relativer Luftfeuchtigkeit Kondensation auftritt. Um diese Bedingungen zuverlässig zu erkennen, ist neben der Messung der Lufttemperatur und -feuchtigkeit durch den DHT11 ein zusätzlicher Sensor erforderlich, der die Temperatur der Windschutzscheibe erfasst.

Ein Algorithmus wertet anschließend alle drei Messgrößen Lufttemperatur, Luftfeuchtigkeit und Scheibentemperatur in regelmäßigen Abständen aus. Liegt beispielsweise eine Lufttemperatur von 10 °C bei einer relativen Luftfeuchtigkeit von 70 % vor, so tritt Kondensation

auf, sobald die Scheibentemperatur auf 4,79 °C oder darunter absinkt (siehe Abbildung 3.3). Der Algorithmus muss folglich nicht nur den aktuellen Zustand erfassen, sondern insbesondere die Entwicklung der Scheibentemperatur überwachen, um rechtzeitig vor dem Erreichen des Taupunkts reagieren zu können.

Als Gegenmaßnahme wird eine leichte Erwärmung der Scheibe vorgesehen. Diese kann entweder über das HVAC-System oder, sofern vorhanden, über eine integrierte Scheibenheizung erfolgen. Ziel ist es, den Temperaturunterschied zwischen Scheibe und Taupunkt so gering zu halten, dass eine Kondensation vermieden wird.

3.4 Kritische Betrachtung und Lösungsansätze

Bei der Betrachtung des vorgestellten Konzepts ergeben sich mehrere Einschränkungen und Herausforderungen. Für Fahrzeuge mit Verbrennungsmotor ist das System nur bedingt geeignet, da ein automatisches Heizsystem die Fahrzeugbatterie unnötig belasten würde. Da die elektrische Energie hier nicht primär für den Fahrzeugantrieb vorgesehen ist, steht der zusätzliche Energieverbrauch in keinem sinnvollen Verhältnis zum Nutzen.

Für Elektrofahrzeuge erscheint der Ansatz grundsätzlich geeigneter, da die benötigte Energie aus dem Hauptakku bezogen werden kann. Dennoch zeigt sich auch hier ein zentrales Problem: Der präventive Charakter des Systems führt in vielen Situationen zu einem ineffizienten Energieeinsatz. Ist das System dauerhaft aktiv, wird kontinuierlich Energie für das Auslesen und Auswerten der Sensordaten sowie für eine potenzielle Scheibenbeheizung verbraucht, selbst wenn kein Beschlag droht. Insbesondere bei längeren Standzeiten, etwa über Nacht oder bei selten genutzten Fahrzeugen, würde das System unnötig Energie verbrauchen und somit die Reichweite eines Elektrofahrzeugs spürbar reduzieren.

3.4.1 Lösungsansatz 1: Aktivierung nur beim Ladevorgang

Ein möglicher Ansatz besteht darin, das System ausschließlich während des Ladevorgangs eines Elektrofahrzeugs zu aktivieren. In diesem Fall hätte der zusätzliche Energieverbrauch keinen direkten Einfluss auf die Reichweite. Allerdings wird dabei weiterhin Energie eingesetzt, auch wenn eine Defogging-Maßnahme nicht erforderlich ist. Der grundsätzliche Nachteil eines ineffizienten Dauerbetriebs bleibt somit bestehen, sodass dieser Ansatz zwar praktikabel, jedoch nicht nachhaltig ist.

3.4.2 Lösungsansatz 2: Intelligentes Defogging

MEine weitere Möglichkeit ist die Implementierung eines intelligenten Systems, das das Fahrverhalten des Nutzers analysiert. Erkennt das System beispielsweise wiederkehrende Fahrzeiten, etwa den täglichen Arbeitsweg, könnte das Defogging automatisch zu diesen Zeiten aktiviert werden, ohne dass ein manuelles Eingreifen erforderlich ist. Dieser Ansatz reduziert zwar unnötigen Energieverbrauch, birgt jedoch ebenfalls Risiken. Ändert sich das Fahrverhalten, etwa durch Urlaub oder flexible Arbeitszeiten, kann das System unnötig aktiv werden und erneut Energie verschwenden.

3.4.3 Lösungsansatz 3: Ferngesteuertes Defogging

Die effektivste Lösung stellt ein ferngesteuertes Defogging-System dar. In diesem Szenario ist das System nicht dauerhaft präventiv aktiv, sondern wird gezielt durch den Fahrer gesteuert. Über eine App könnten Abfahrtszeiten manuell festgelegt oder wiederkehrende Routinen definiert werden. Das System würde dann wenige Minuten vor der geplanten Abfahrt aktiviert, um den aktuellen Zustand der Scheiben zu überprüfen und bei Bedarf geeignete Gegenmaßnahmen einzuleiten.

Auf diese Weise wird Energie ausschließlich dann verbraucht, wenn ein tatsächlicher Bedarf besteht. Gleichzeitig vereinfacht sich der erforderliche Algorithmus, da nicht mehr die langfristige Entwicklung der Scheibentemperatur überwacht werden muss, sondern lediglich der aktuelle Zustand relevant ist. Dieser Ansatz stellt somit einen sinnvollen Kompromiss zwischen Funktionalität und Energieeffizienz dar und bietet die praktikabelste Lösung zur Umsetzung des vorgestellten Konzepts.

4 Fazit und Ausblick

Ziel dieser Arbeit war es, die Grundlagen eines präventiven Defogging-Systems auf Basis eines ESP32-Mikrocontrollers und eines DHT11-Sensors zu untersuchen sowie eine mögliche theoretische Implementierung zu entwerfen. Hierzu wurden zunächst die relevanten technischen Grundlagen erläutert, insbesondere die Funktionsweise des Mikrocontrollers und die Eigenschaften des eingesetzten Sensors. In der praktischen Umsetzung wurde ein Prototyp aufgebaut und eine Messreihe durchgeführt, um die Messgenauigkeit des DHT11 zu bewerten. Aufbauend auf diesen Ergebnissen konnte anschließend ein theoretisches Konzept für einen Defogging-Algorithmus entwickelt werden.

Die Messergebnisse zeigen, dass der DHT11 grundsätzlich geeignet ist, Temperatur- und Luftfeuchtigkeitsverläufe zuverlässig zu erfassen. Insbesondere bei der relativen Luftfeuchtigkeit treten jedoch messbare Abweichungen auf. Für das entwickelte Konzept bedeutet dies, dass das Defogging-System unter Umständen geringfügig zu früh oder zu spät aktiviert werden könnte, was im praktischen Einsatz jedoch nur einen begrenzten Einfluss hätte.

Das vorgestellte Konzept weist mehrere Einschränkungen auf. So kann der DHT11 Temperaturen unter 0 °C nicht erfassen und ist damit für Anwendungen im Bereich der Scheibenenteisung ungeeignet. Zudem ist der Energieverbrauch eines dauerhaft aktiven präventiven Systems insbesondere bei Verbrennerfahrzeugen kritisch zu bewerten, da die Fahrzeugbatterie unnötig belastet würde. Für Elektrofahrzeuge erscheint der Ansatz grundsätzlich geeigneter, jedoch führt auch hier der präventive Betrieb zu einem erhöhten Energiebedarf.

Für zukünftige Arbeiten bietet es sich an, das Konzept mit präziseren Sensoren wie dem DHT22 oder industriellen Temperatur- und Feuchtigkeitssensoren zu erweitern und den Algorithmus praktisch umzusetzen, um reale Reaktionszeiten und den tatsächlichen Energieverbrauch zu analysieren. Darüber hinaus könnte eine Integration in bestehende Fahrzeugsysteme untersucht werden, um vorhandene Klima- und Lüftungsfunktionen effizienter zu nutzen. Insgesamt zeigt diese Arbeit, dass ein präventives Defogging technisch realisierbar ist, jedoch in der vorgestellten Form nicht den ursprünglich angestrebten Anforderungen an die Energieeffizienz genügt. Ein ferngesteuertes Defogging-System stellt in diesem Zusammenhang eine praktikable und energieeffizientere Alternative dar.

Literatur

[1] Herbert Bernstein. *Messelektronik und Sensoren: Grundlagen der Messtechnik, Sensoren, analoge und digitale Signalverarbeitung*. Springer Vieweg, 2013. ISBN: 978-3658005481.

[2] *DHT22/AM2302 Digitaler Temperatur- und Luftfeuchtigkeitssensor*. n.d. URL: https://www.elektronik-kompendium.de/sites/praxis/bauteil_dht22.html. (zuletzt geprüft: 14.12.2025).

[3] A Ene und C Teodosiu. „Studies dealing with defogging and de-icing phenomena on vehicles' windshield: a review". In: *IOP Conference Series: Earth and Environmental Science* 664.1 (2021).

[4] *ESP32 Grundlagen*. 2023. URL: https://digitalewelt.at/esp32-grundlagen. (zuletzt geprüft: 14.12.2025).

[5] *Hochschule Hamm Lippstadt: Temperatur und Feutchtigkeitssensor DHT11*. 2023. URL: https://wiki.hshl.de/wiki/index.php/Temperatur-_und_Feuchtigkeitssensor_DHT11. (zuletzt geprüft: 14.12.2025).

[6] Martin Kappes. *Netzwerk- und Datensicherheit: Eine praktische Einführung*. Springer Vieweg, 2013. ISBN: 978-3834806369.

[7] Korff Superwand. *Taupunktabelle*. pdf online. (zuletzt geprüft: 14.12.2025). n.d.

[8] *WAs ist das OSI-Modell?* 2025. URL: https://www.ibm.com/de-de/think/topics/osi-model. (zuletzt geprüft: 14.12.2025).

[9] *Was ist ein Mikrocontroller*. 2025. URL: https://www.ibm.com/de-de/think/topics/microcontroller. (zuletzt geprüft: 14.12.2025).

[10] *Wie Sensoren die Reichweite von E-Autos steigern*. 2024. URL: https://www.all-electronics.de/e-mobility-reichweite/wie-sensoren-die-reichweite-von-e-autos-steigern/794568. (zuletzt geprüft: 14.12.2025).

Anhang

A. Code

B. Abbildungen

A. Code Der verwendete Code zum Auslesen der DHT11 Sensoren mit dem ESP32 und zum anzeigen der Messwerte auf einer Website.

```
#include <Arduino.h>
#include "DHT.h"
#include <WiFi.h>

#define DHTPIN 14
#define DHTTYPE DHT11

const char* ssid = "iPhone VBackhaus";
const char* password = "wtIhe357";

DHT dht(DHTPIN, DHTTYPE);
WiFiServer server(80);

void setup() {
Serial.begin(115200);

WiFi.begin(ssid, password);
Serial.println("Verbindet mit WLAN...");
while (WiFi.status() != WL_CONNECTED) {
  delay(1000);
  Serial.println("Noch keine Verbindung...");
}

Serial.println("Verbunden");
Serial.print("IP-Adresse: ");
Serial.println(WiFi.localIP());

dht.begin();
server.begin();
Serial.println("Server gestartet");
}

void loop() {
WiFiClient client = server.available();
if (!client) return;

Serial.println("Neuer Client verbunden");
String request = client.readStringUntil('\r');
Serial.println(request);
client.flush();

//pruefen ob die Anfrage nach "/data" geht
if (request.indexOf("GET /data") >= 0) {
  float h = dht.readHumidity();
```

```
      float t = dht.readTemperature();

      client.println("HTTP/1.1 200 OK");
      client.println("Content-Type: application/json");
      client.println("Connection: close");
      client.println();

      if (isnan(h) || isnan(t)) {
      client.println("{\"error\": \"Sensorfehler\"}");
      } else {
      client.printf("{\"temperature\": %.2f, \"humidity\": %.2f}", t, h);
      }
      return;
    }

    //HTML datei mit Script
    String html = R"rawliteral(
    <!DOCTYPE html>
    <html>
    <head>
      <meta charset="UTF-8">
      <title>DHT11 Sensor</title>
    </head>
    <body>
      <h1>ESP32 DHT11 Sensorwerte</h1>
      <p>Temperatur: <span id="temp">--</span> C</p>
      <p>Luftfeuchtigkeit: <span id="hum">--</span> %</p>

      <script>
      // Diese Funktion holt neue Daten vom ESP32
      async function ladeDaten() {
        const antwort = await fetch('/data');      // Anfrage an /data schicken
        const daten = await antwort.json();        // Antwort als JSON lesen
        document.getElementById('temp').textContent = daten.temperature;
        document.getElementById('temp').textContent = daten.temperature;
      }

      ladeDaten();                 // Beim Laden der Seite einmal aufrufen
      setInterval(ladeDaten, 3000); // Alle 3 Sekunden automatisch neu laden
      </script>
    </body>
    </html>
```

```
    )rawliteral";

    client.println("HTTP/1.1 200 OK");
    client.println("Content-type:text/html");
    client.println("Connection: close");
    client.println();
    client.println(html);
    client.println();
    }
```

Listing 4.1: Code zur Auslesung des DHT11 Sensors mit dem ESP32

B. Abbildungen

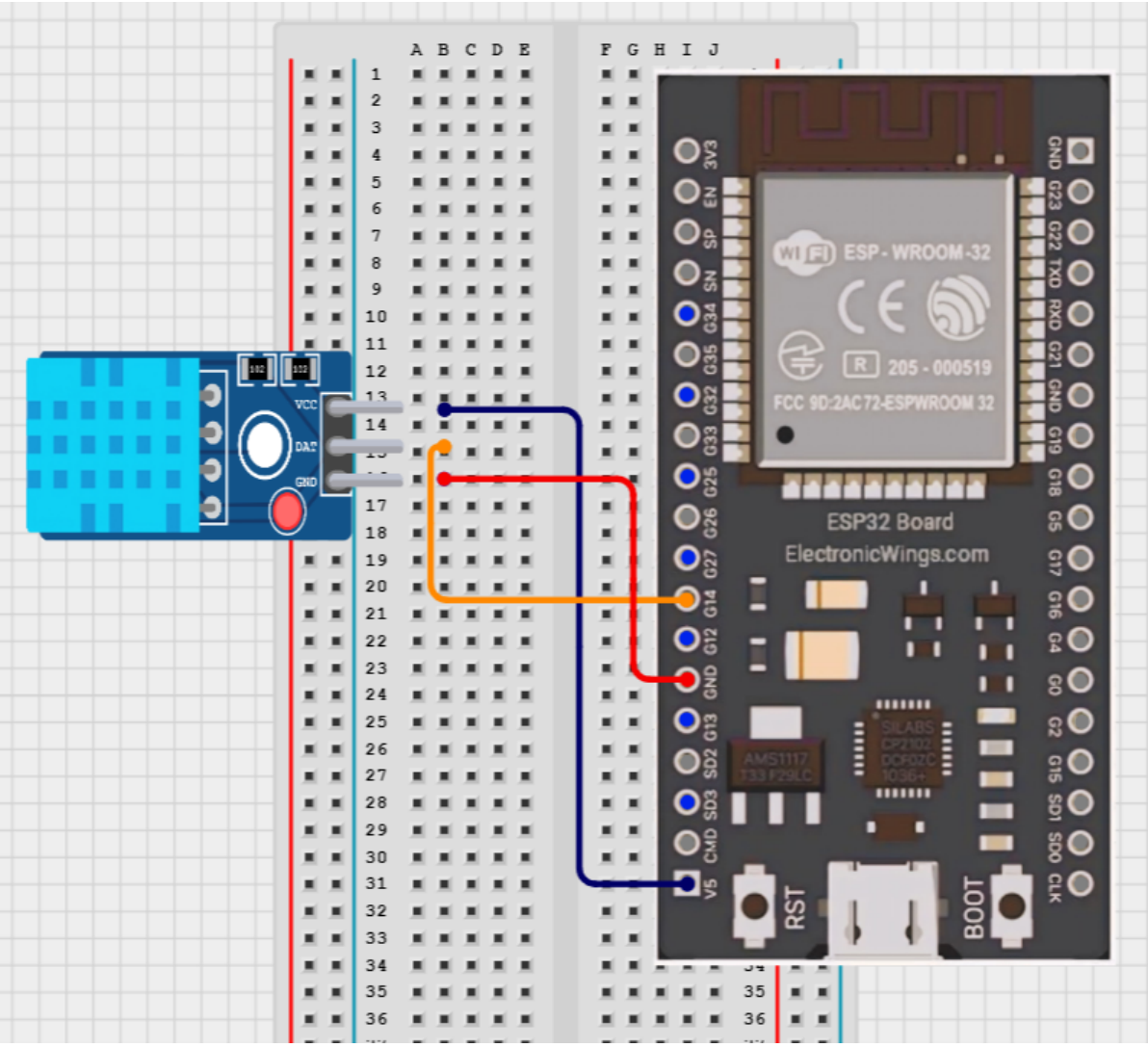


Abbildung 4.1: Anhang: Schaltplan DHT11 (eigene Abbildung)

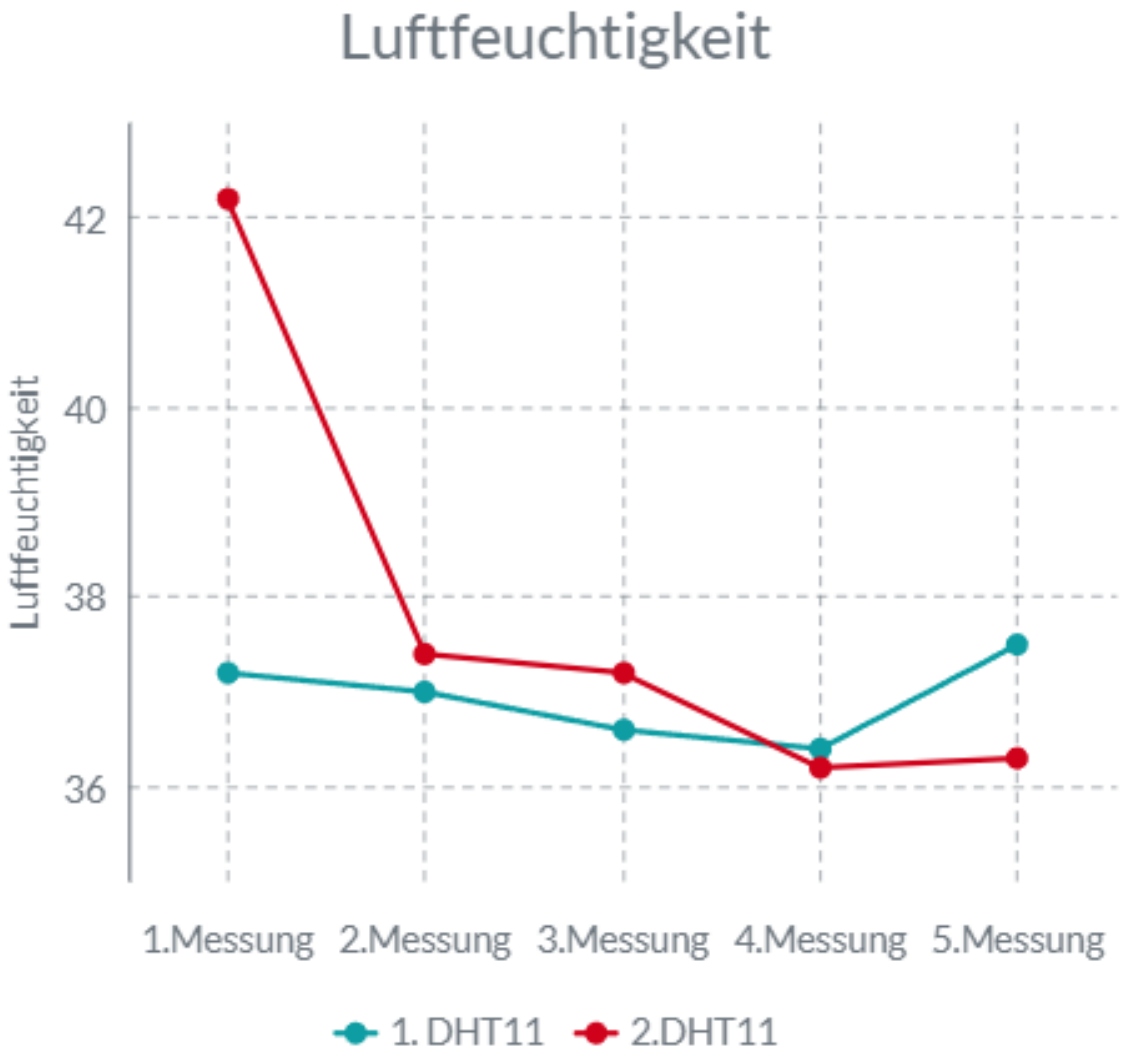


Abbildung 4.2: Anhang: Luftfeuchtigkeitsmessreihe

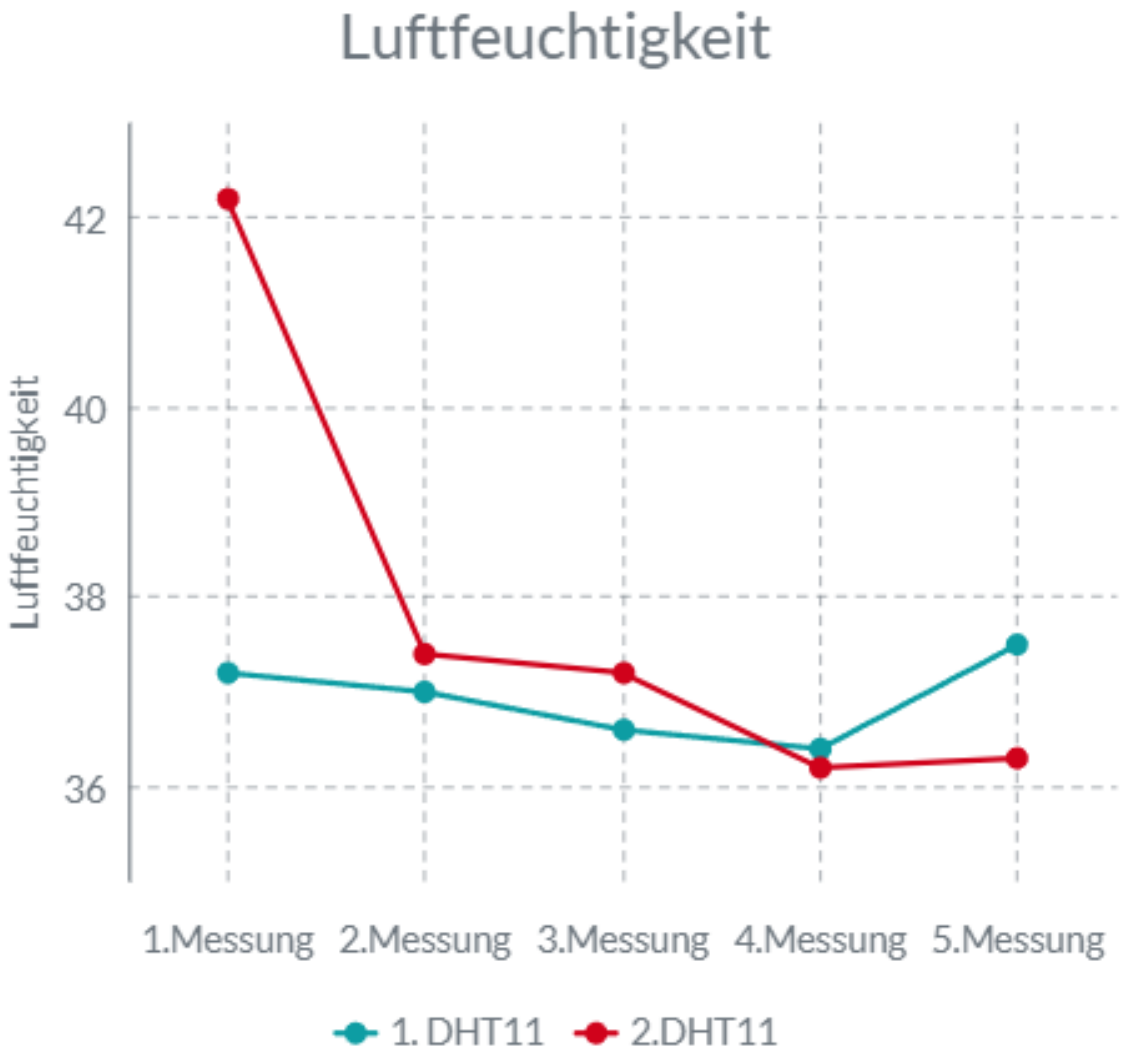


Abbildung 4.3: Anhang:Temperaturmessreihe

Luft-Temp.	Taupunkttemperaturen in °C bei einer relativen Luftfeuchtigkeit von										
C°	45%	50%	55%	60%	65%	70%	75%	80%	85%	90%	95%
2°C	-7.77	-6.56	-5.43	-4.40	-3.16	-2.48	-1.77	-0.98	-0.26	0.47	1.20
4°C	-6.11	-4.88	-3.69	-2.61	-1.79	-0.88	-0.09	0.78	1.62	2.44	3.20
6°C	-4.49	-3.07	-2.10	-1.05	-0.08	0.85	1.86	2.72	3.62	4.48	5.38
8°C	-2.69	-1.61	-0.44	0.67	1.80	2.83	3.82	4.77	5.66	6.48	7.32
10°C	-1.26	0.02	1.31	2.53	3.74	4.79	5.82	6.79	7.65	8.45	9.31

Abbildung 4.4: Anhang: Auschnitt aus der Taupunkttabelle ([7], S.1)